

Message Queues

• A message queue enables asynchronous communication between services. Producer sends message → Consumer processes message later.

* Queue is used (FIFO). $\begin{matrix} \text{enqueue(message)} \\ \text{dequeue(message)} \end{matrix} \rightarrow \text{processed in order.}$

* Priority Queue (Heap). Used when tasks have priority.

Ex!:- • fraud alerts • system critical jobs. (Job schedulers)

Ex!:- In Uber, when user requests a ride :-

User → API → Ride Services

(Ride services do many tasks :-

- find nearby drivers
- send driver notifications
- Update analytics
- Log events

(everything should be asynchronous)

* Service Decoupling (Service A → Queue → Service B)

↓ A can continue even if B is slow

* Traffic Buffering (Queue absorbs spikes).

Ex!:- Sale on Amazon • 10K orders/sec → queue buffers tasks while workers process gradually.

* Queues store messages until processed. If worker crashes, message stays in queue.

Kafka

* Kafka is a distributed event streaming platform that stores messages in append only logs for high throughput data pipelines. Consider Netflix:-

- Millions of events happen per second :-
 - video start
 - pause
 - buffering
- Netflix must process these events for :-
 - recommendation
 - monitoring
 - analytics.

Architecture

Services → Kafka → stream processors → Databases/~~for~~ Analytics.

* Kafka is used for event streaming, log pipelines, real-time analytics and it has durable storage & high throughput.

Working

* Messages are stored in topics. Topics are divided into partitions. Consumers read partitions sequentially.

Data structure Used

- * ~~Append~~
- * Append only Log :- append (message) → stored sequentially on disk.
read (offset)
- * Partition Hashing :- To distribute messages across partitions,
$$\text{partition} = \text{hash}(\text{key}) \% \text{partitions}$$

Similar to consistent hashing concept.
- * Offset Index (Allows replay & reprocessing)
offset → message location.

LinkedIn → created Kafka

Netflix → Event streaming

Uber → Ride & location pipelines

Spotify → Real time analytics.

Rabbit MQ

- * A message broker designed for reliable task queues.
Commonly used for :-
 - background jobs
 - worker queues
 - task processing
- Compared to Kafka, RabbitMQ focuses more on task delivery guarantees.

Architecture

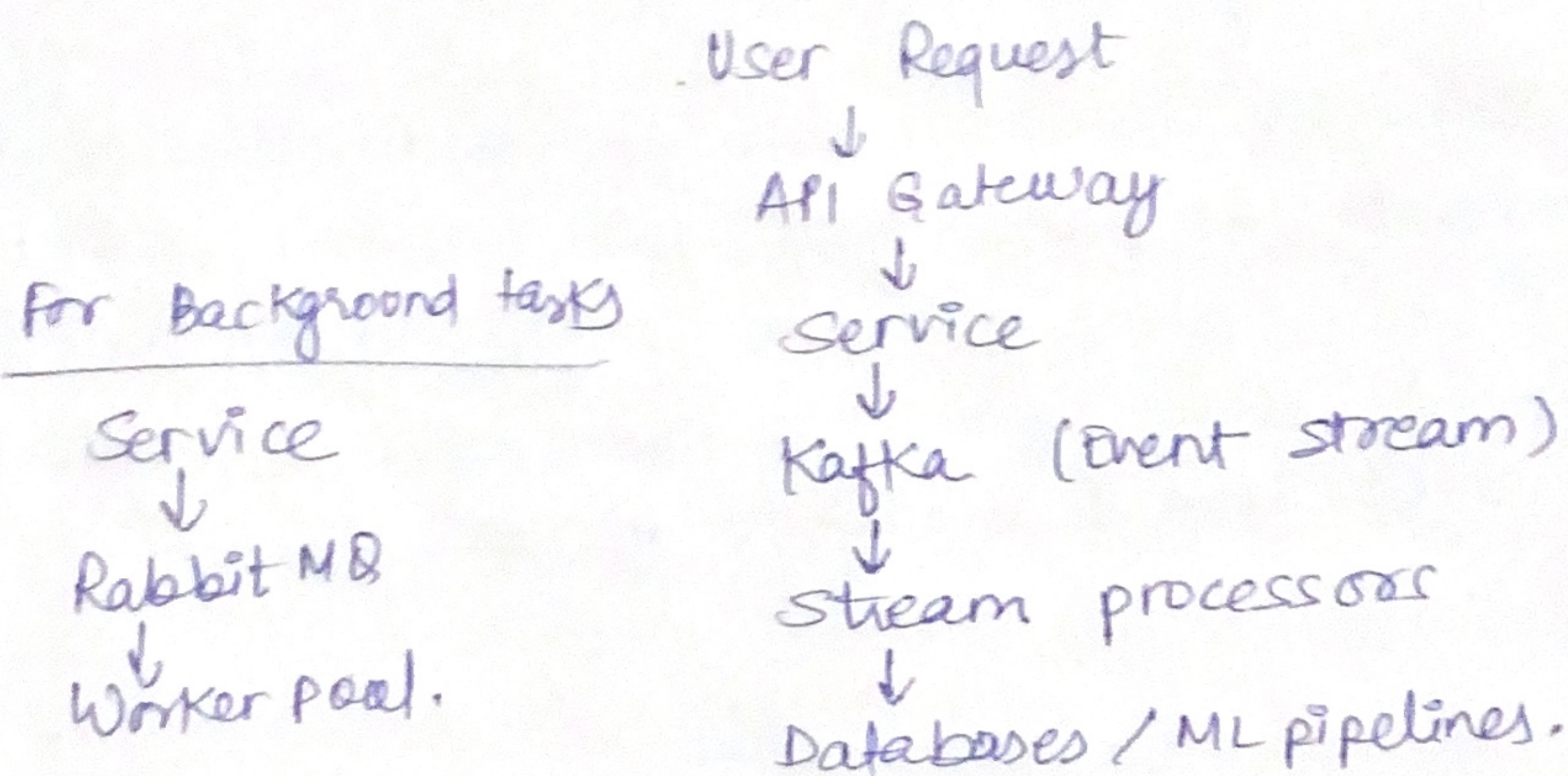
Producers → Exchange → Queue → Consumer.

Data Structure :- Except queues it uses Routing Tables.

Exchanges using routing keys :-

routing_Key → queue mapping
Internally implemented using hashmaps / routing tables.

Modern System (Large Platform)



* Many companies run both.

Publish Subscribe Model

- * Pub-Sub model is a messaging pattern where producers (publishers) send events to messaging system & multiple subscribers (consumers) receive those events independently.

Why it is needed

Pub-sub enables loose coupling between services. A service simply publishes an event like 'order-created' and any no. of downstream systems can react without changing the original service.

Working

- * A publisher generates an event (new order)
- * Event is sent to a broker (topic or event channel)
- * Multiple subscribers listen to that topic.
- * Each subscriber processes the event independently.

Production Use Case

- * Netflix uses pub-sub pipelines for user activity events (play, pause, search) that feed analytics, recommendation engines, monitoring systems simultaneously.
- * Spotify streams listening events through pub-sub pipelines to update recommendations & real time dashboards.

Trade offs

Advantages

- Services remain loosely coupled
- Easy to add new consumers without modifying producers.
- Supports real time event distribution.

Disadvantages

- Harder debugging because events flow asynchronously.
- Requires careful monitoring of message delivery. & ordering guarantees can be complex.

Event-Driven Architecture

- EDA is a design approach where services communicate by producing and reacting to events rather than calling each other synchronously.

Event represents something that has happened in system.
(payment_completed, user_registered).

Why it is needed?

- Helps to scale & evolve. When one component generates an event other components can respond independently without blocking the original process. This allows system to process high volume of activity while remaining flexible.

Working

- * A service performs an action and emits an event.
- * The event is stored or streamed through an event broker.
- * Other services subscribe to the event stream.
- * Each service processes the event according to its responsibility.

Production use case

- * Amazon uses EDA in order processing. When an order is placed, multiple services react to the event to handle payment, shipping, notification & inventory updates.
- * Uber relies on event-driven pipeline for ride events such as driver location updates, ride requests & billing triggers.

Tradeoffs

Advantages

- Highly scalable systems
- Loose coupling between services
- Real-time processing of large event streams.

Disadvantage

- Eventual consistency between services.
- Harder tracing & debugging
- Requires robust event driven management & monitoring.

Database as Queue

- * Using a database table to store & process tasks like queue. Instead of a message broker, you use a table:-

Jobs

- id
- status
- payload
- created_at

} Workers pick jobs from this table.

- * Small systems often need background processing (emails, jobs, etc.) but setting up Kafka / RabbitMQ may feel heavy.
- Simple setup & good for low-scale systems.

Working

- 1) Service inserts a job into DB
- 2) Worker polls DB for pending jobs
- 3) Worker locks a row (to avoid duplicate processing)
- 4) Processes job → Updates status.

- * GitHub (earlier architecture) :- Used DB-backed job systems before scaling.

- * Pros:-
 - Very simple
 - Easy debugging
 - No extra infra

- * Cons:-
 - Poor scalability
 - DB becomes bottleneck
 - Polling is inefficient.
 - Not suitable for high throughput.

AI Task Queues

- * Queues designed to manage and schedule AI-related workloads like LLM calls, embedding generation & batch ~~operation~~ processing.

Why it is needed?

- * AI system needs:-
 - Workload buffering
 - Retry handling
 - Prioritization
 - async execution
 - cost control

Working

- 1) User uploads document → system pushes task to queue
- 2) Worker picks task → Generate embedding & summary
- 3) Stores result.

Architecture

API → Queue → Worker → LLM / Embedding Model → DB

Production Use case

- Open AI / Anthropic clients
Queue embedding jobs instead of blocking API.
- Notion AI : Process document summarisation asynchronously.

Common Tasks in AI Queues

- Embedding generation
- document chunking
- summarization
- fine tuning jobs
- batch inference
- re-indexing vector DB.

Trade offs

- prevents API blocking
- improves scalability
- enables batching
- better cost control

- adds latency
- queue management complexity
- monitoring required.